



Inteligencia Artificial
Ingeniería en Sistemas de Información
Año 2026

Trabajo Práctico N° 02

Entrega Final

GRUPO N° 19

Apellido y Nombres	Dirección de E-Mail
Gliozzo, Tomás Esteban	tgliozzo@frba.utn.edu.ar
Gobbi, Micaela Nicole	mgobbi@frba.utn.edu.ar
Rocca, Cecilia Inés	crocca@frba.utn.edu.ar
Neustadt, Agustín Tiago	aneustadt@frba.utn.edu.ar
Elias, Yasmin Laina	yelias@frba.utn.edu.ar
Riolfi, Ignacio Joaquín	iriolfi@frba.utn.edu.ar
Cubelli, Leonel Oscar	lcubelli@frba.utn.edu.ar

Fecha de Presentación

31/05/2026

Universidad Tecnológica Nacional
Facultad Regional Buenos Aires



UTN.BA

FACULTAD
REGIONAL
BUENOS AIRES

Trabajo Práctico de Sistemas Inteligentes - RNA: Predicción de Popularidad de Canciones en Spotify mediante Redes Neuronales Artificiales

Tomás Esteban Gliozzo, Micaela Nicole Gobbi, Cecilia Inés Rocca,
Agustín Tiago Neustadt, Yasmin Laina Elías, Ignacio Joaquín Riolfi,
Leonel Oscar Cubelli

Universidad Tecnológica Nacional. Facultad Regional Buenos Aires.
Buenos Aires. Argentina

Resumen. El presente trabajo práctico implementa un Sistema Inteligente basado en una Red Neuronal Artificial Multi-Perceptrón con entrenamiento Backpropagation (RNA MLP BP) para predecir el nivel de popularidad de canciones del género musical clásico en la plataforma Spotify. A partir de un dataset de 2.394 piezas clásicas filtradas de un corpus de más de 114.000 canciones, se entrenaron tres modelos utilizando 14 atributos acústicos y estructurales como variables de entrada: dos orientados a regresión continua y uno a clasificación multiclase en tres categorías (Baja, Media y Alta). Se analizan las métricas obtenidas, las limitaciones del dataset y las posibles mejoras futuras que se pueden plantear a este Sistema Inteligente.

Palabras clave: *red neuronal artificial, backpropagation, multi-perceptrón, Spotify, popularidad musical, música clásica, clasificación multiclase, regresión, aprendizaje supervisado, Keras, TensorFlow.*

1. Introducción

1.1. Descripción del problema

El presente trabajo práctico tiene como objetivo desarrollar un Sistema Inteligente basado en Redes Neuronales Artificiales (*RNA*) capaz de predecir el nivel de popularidad de canciones del género musical “*clásico*” o “*música académica*” en la plataforma Spotify, en función de sus características acústicas, estructurales y metadatos.

Spotify asigna a cada canción un puntaje de popularidad entre 0 y 100, basado en la cantidad y frecuencia de sus reproducciones recientes. Este indicador es clave para sellos discográficos, artistas y equipos de marketing, ya que determina la visibilidad de una pieza dentro de la plataforma (*listas editoriales, recomendaciones, radio*).

A diferencia del rock, pop u otros géneros populares, la música clásica presenta características muy particulares: alta proporción de instrumentalidad, mayor duración promedio en las canciones, menores valores de *loudness* y *danceability*, y una distribución de popularidad con media en torno a 34,6 sobre 100. Estos factores hacen especialmente desafiante la predicción de popularidad en este segmento.

El problema consiste en determinar, a partir de 14 atributos numéricos de una pieza clásica, si esta tendrá popularidad Baja, Media o Alta dentro de la plataforma o el valor de popularidad según el método de entrenamiento elegido.

1.2. Relevancia

La industria de la música clásica, aunque más pequeña que otros géneros, moviliza importantes recursos en grabaciones, distribuciones digitales y conciertos en vivo. Un sistema capaz de anticipar la recepción de una pieza clásica en plataformas de streaming permitiría:

- **Optimizar** decisiones de grabación y producción (*selección de obras, directores, intérpretes*).
- **Orientar** estrategias de marketing y posicionamiento en plataformas digitales.
- **Ayudar** a *curadores de playlists*¹ en la selección de piezas con mayor potencial de engagement.
- **Mejorar** los algoritmos de recomendación de música clásica para audiencias especializadas.

¹ persona o equipo encargado de seleccionar, organizar y actualizar canciones dentro de una lista de reproducción en plataformas digitales como Spotify

1.3. Ejemplo ilustrativo

A continuación se muestran tres ejemplos del dataset utilizado para facilitar la comprensión del problema:

<i>Param\Pista</i>	<i>Sogno D'or</i>	<i>Gnossienne No. 1</i>	<i>I Ain't Worried</i>
<i>danceability</i>	0.221	0.376	0.704
<i>energy</i>	0.203	0.00579	0.797
<i>acousticness</i>	0.991	0.996	0.0826
<i>loudness</i>	-14.063	-31.615	-5.927
<i>tempo</i>	152.553	71.827	139.994
<i>Popularidad</i>	33	66	96

2. Elementos del trabajo y metodología

2.1. Descripción de los datos

El dataset fue obtenido de Kaggle (*Spotify Tracks Dataset, Maharshi Pandya*)² y contiene información de más de 114.000 canciones de múltiples géneros. Tras filtrar exclusivamente el género “classical” (*música clásica*) y eliminar registros con popularidad igual a 0 o sin datos válidos, se obtuvo un subconjunto de 2.394 canciones clásicas con popularidades entre 1 y 96.

2.1.1. Variables de entrada (14 atributos)

<i>Variable</i>	<i>Tipo</i>	<i>Descripción</i>
danceability	Núm. continua [0–1]	Aptitud de la pista para el baile
energy	Núm. continua [0–1]	Intensidad y actividad percibida
acousticness	Núm. continua [0–1]	Nivel acústico vs. sintético
instrumentalness	Núm. continua [0–1]	Probabilidad de ausencia de voces

² <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

liveness	Núm. continua [0–1]	Probabilidad de que sea en vivo
valence	Núm. continua [0–1]	Positividad emocional de la pista
speechiness	Núm. continua [0–1]	Presencia de palabras habladas
loudness	Núm. continua (dB)	Volumen promedio (valores negativos)
tempo	Núm. continua (BPM)	Velocidad en pulsos por minuto
duration_ms	Núm. entera (ms)	Duración de la pista en milisegundos
time_signature	Entera [0–5]	Firma de tiempo (compás)
explicit	Booleana [0/1]	Contiene lenguaje explícito
mode	Booleana [0/1]	Escala mayor (1) o menor (0)
key	Entera [0–11]	Nota musical base (Do=0, Do#=1, ...)

2.1.2. Variable objetivo

La variable objetivo es *popularity*, un entero de 1 a 100. La media del dataset clásico es 34,6, lo que indica que la mayoría de las piezas clásicas tienen popularidad media tirando a baja en Spotify. Su distribución es asimétrica, con escasa representación en valores altos (>66), lo que condiciona directamente la capacidad del modelo para aprender la clase Alta.

2.1.3. Estadísticas descriptivas del dataset

<i>Estadística</i>	<i>Cantidad</i>	<i>Media</i>	<i>Desv. Est.</i>	<i>Min.</i>	<i>Máx.</i>
popularity	2.394	34,65	19,35	1	96
danceability	2.394	0,393	0,148	0	0,803
energy	2.394	0,29	0,252	0,001	0,983
acousticness	2.394	0,789	0,302	0	0,996
instrumentalness	2.394	0,402	0,434	0	0,982

2.2. Preparación de los datos

Se aplicaron las siguientes técnicas de preparación:

- **Filtrado por género:** se seleccionaron únicamente canciones etiquetadas como “classical” en la columna **track_genre**.
- **Limpieza:** se eliminaron registros con valores nulos y popularidades iguales a 0.
- **Corrección de formato:** el archivo CSV presentaba números decimales con coma como separador, por lo que se preprocesaron los datos antes de ser cargados con Panda.
- **Normalización:** se aplicó StandardScaler a las 14 variables de entrada, centrándolas en media 0 y desviación estándar 1. Esto es fundamental para que el gradiente descienda correctamente en la RNA.
- **Separación de conjuntos:** El 70 % (1.675 muestras) se usó para entrenamiento/validación y el 30 % (719 muestras) para prueba final.
- **Discretización:** Para uno de los modelos planteados, se dividió la clase popularidad en 3 categorías o clases: Baja (0–33), Media (34–66), Alta (67–100).

2.3. Distribución de los datos

Conjunto	Cantidad de muestras	%
Entrenamiento + Validación interna	1.675	70 %
Prueba (Test)	719	30 %
Total	2.394	100 %

Dentro del conjunto de entrenamiento, el modelo utiliza un 20 % adicional como validación interna (`validation_split = 0,2`) para monitorear el loss en cada época sin contaminar el conjunto de prueba.

2.4. Herramientas y librerías utilizadas

Herramienta / Librería	Uso
Python 3.12	Lenguaje principal
TensorFlow / Keras 2.x	Construcción y entrenamiento de las RNA
Scikit-learn 1.x	Normalización, split de datos y métricas
Pandas 2.x	Carga y manipulación del dataset
NumPy 1.x	Operaciones numéricas
Matplotlib 3.x	Gráficos de loss y matrices de confusión
Google Colab	Entorno de ejecución en la nube
Google Drive	Almacenamiento del dataset CSV
Anthropic: Claude Sonnet 4.6	Asistente de codificación basado en IA
Google: Gemini 3.1 Pro	Asistente de codificación basado en IA

3. Resultados

Se realizaron tres entrenamientos con diferentes arquitecturas y parámetros. Los dos primeros corresponden al Modelo A (regresión continua) y el tercero al Modelo B (clasificación multiclase). A continuación, se desarrollarán los distintos entrenamientos realizados para cada modelo y sus resultados:

3.1. Entrenamiento 1

3.1.1. Modelo, arquitectura y parámetros

Parámetro	Valor
Tipo de modelo	Regresión continua (Modelo A)
Capas ocultas	(64, ReLU) → (32, ReLU) → (16, ReLU)
Capa de salida	(1, lineal)
Optimizador	Adam (learning_rate = 0,01)
Función de pérdida	Mean Squared Error (MSE)
Épocas	200 (<i>paró en 51</i>)

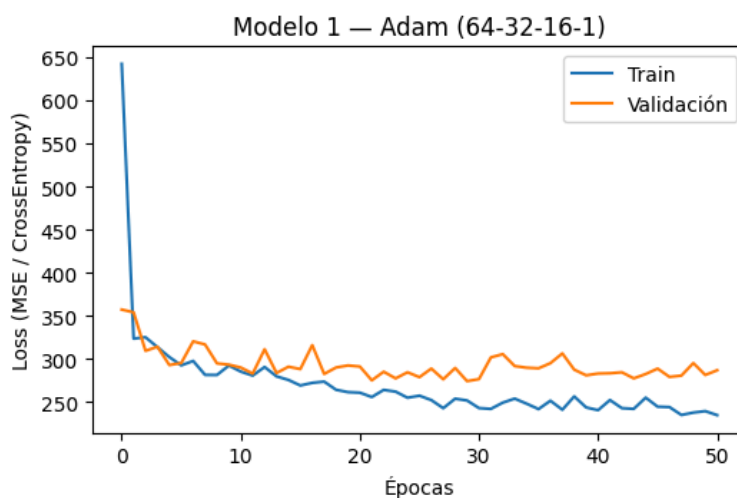
Batch size	32
Normalización	StandardScaler

3.1.2. Justificación

Teníamos 14 variables de entrada con naturalezas y escalas muy distintas (*por ejemplo, el tempo en pulsos por minuto vs. el nivel de energía entre 0 y 1*). Por lo que vimos en la cursada, usar un optimizador como Adam nos venía bien porque adapta la tasa de aprendizaje automáticamente, evitando que el entrenamiento se estanque en los primeros pasos.

Armamos la red en forma de "embudo" ($64 \rightarrow 32 \rightarrow 16$ neuronas) buscando que las primeras capas capten los patrones más complejos del dataset y las siguientes vayan resumiendo la información. Usamos ReLU en las capas ocultas para evitar que el gradiente se desvanezca al hacer el backpropagation, y para la salida dejamos una capa lineal, ya que estábamos buscando un número continuo. Finalmente, la normalización (*StandardScaler*) fue obligatoria para asegurar que variables con valores enormes (*como la duración de la pista en milisegundos*) no opaquen a las más chicas durante el ajuste de pesos.

3.1.3. Gráfico de evolución del error (Loss)



3.1.4. Métricas obtenidas

Conjunto	MAE	Error Relativo	R ²
Entrenamiento	11.92	34.27%	0.3872
Prueba (Test)	12.27	35.74%	0.3476

El modelo obtuvo un MAE de 11,92 en entrenamiento y de 12,27 en prueba, lo que indica que en promedio el modelo se aleja aproximadamente 12 puntos del valor real de popularidad. La diferencia entre ambos conjuntos es pequeña, por lo que no se evidencia un sobreajuste significativo.

Sin embargo, el error relativo resulta elevado en ambos casos, lo que sugiere que el modelo tiene dificultades a la hora de predecir de forma precisa la popularidad de una canción. Esto se ve reforzado por el coeficiente de determinación R², que alcanza apenas 0,39 en entrenamiento y 0,35 en prueba, indicando que el modelo explica menos de la mitad de la varianza de los datos.

En conclusión, estas métricas señalan que el modelo logra una aproximación razonable pero con margen de mejora considerable, tanto en capacidad predictiva como en generalización.

3.2. Entrenamiento 2

3.2.1. Modelo, arquitectura y parámetros

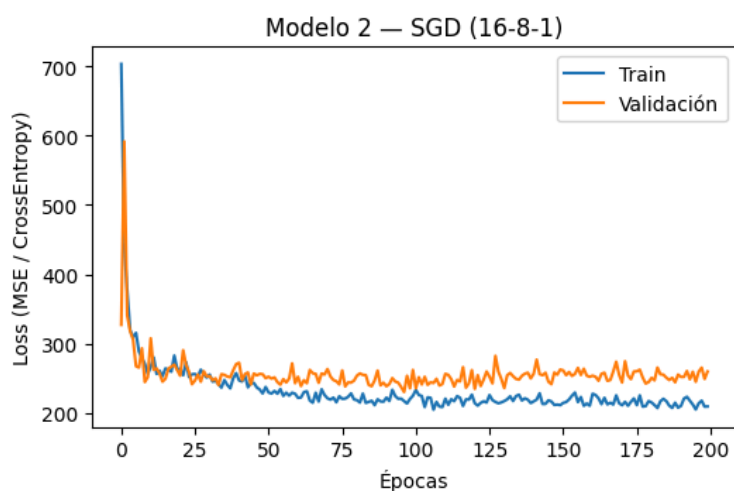
Parámetro	Valor
Tipo de modelo	Regresión continua (Modelo A, variante)
Capas ocultas	(16, ReLU) → (8, ReLU)
Capa de salida	(1, lineal)
Optimizador	SGD (learning_rate = 0,001, momentum = 0,9)
Función de pérdida	Mean Squared Error (MSE)
Épocas	200 (paró en 200)
Batch size	32
Normalización	StandardScaler

3.2.2. Justificación

Este modelo de entrenamiento se eligió con el objetivo de evaluar el impacto de cambiar tanto el optimizador como la arquitectura respecto al modelo anterior. Se optó por SGD con momentum $(0,9)$ como alternativa a Adam.

El **learning rate** elegido es más conservador que el utilizado anteriormente, lo que implica actualizaciones más pequeñas y un aprendizaje más gradual por parte del modelo. La arquitectura se redujo a dos capas ocultas ($16 \rightarrow 8$ neuronas), significativamente más compacta que la del Modelo A, con el objetivo de evaluar si una red más simple es capaz de capturar los patrones relevantes del problema.

3.2.3. Gráfico de evolución del error (Loss)



3.2.4. Métricas obtenidas

Conjunto	MAE	Error Relativo	R ²
Entrenamiento	10.86	31.24%	0.4458
Prueba (Test)	11.64	33.90%	0.3861

El modelo obtuvo un MAE de 10,86 en entrenamiento y 11,64 en prueba, valores similares a los del Entrenamiento 1, aunque el R^2 aumentó a 0,44 y 0,38 respectivamente, lo que indica una capacidad explicativa algo mayor. La diferencia moderada entre entrenamiento y prueba sugiere que no hay sobreajuste severo, pero sí una leve ganancia de generalización.

El error relativo se mantiene elevado (31,24% y 33,90%), lo que confirma que el modelo sigue teniendo dificultades para predecir con precisión el valor de

popularidad. A pesar de que el MAE es comparable al modelo anterior, el R^2 más alto revela que esta arquitectura más compacta y el uso de SGD capturan más variabilidad del target. Esto sugiere que, para este problema, SGD con learning rate conservador y una red reducida ($16 \rightarrow 8$) resulta más eficiente que Adam con una arquitectura más profunda.

3.3. Entrenamiento 3

3.3.1. Modelo, arquitectura y parámetros

Parámetro	Valor
Tipo de modelo	Clasificación multiclase (Modelo B, 3 clases)
Clases	Baja (1–33), Media (34–66), Alta (67–100)
Capas ocultas	(16, ReLU) $\rightarrow$ (8, ReLU) $\rightarrow$ (8, ReLU)
Capa de salida	(3, Softmax)
Optimizador	Adam (learning_rate = 0,01)
Función de pérdida	Sparse Categorical Crossentropy
Épocas	200 (<i>paró en 60</i>)
Batch size	32
Normalización	StandardScaler

3.3.2. Justificación

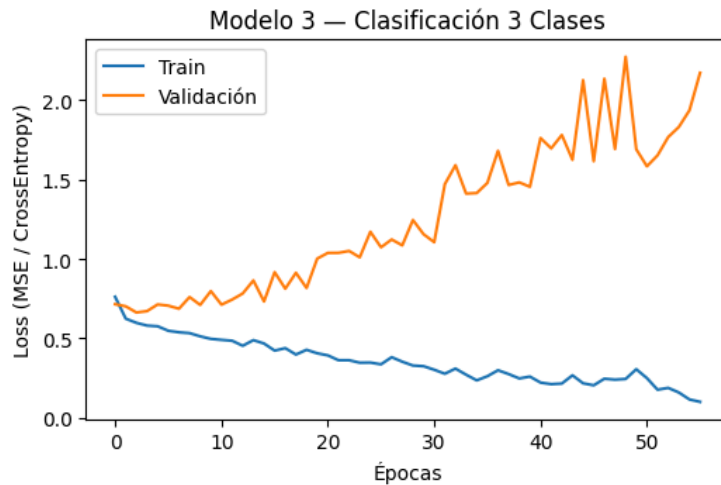
Tras analizar el desempeño de los modelos de regresión y la alta dispersión de los errores observados, se concluyó que la estimación de un valor continuo de popularidad resulta ineficiente debido a la variabilidad inherente a la industria musical. Por consiguiente, se optó por reformular la variable objetivo y abordar el problema mediante un enfoque de clasificación multiclase.

Si bien el planteo de la **entrega parcial** contemplaba una discretización en cinco categorías, el análisis de la distribución del *dataset* de música clásica evidenció una marcada asimetría, dejando a las clases extremas ("*Muy Baja*" y "*Muy Alta*") severamente subrepresentadas. Para mitigar este desbalance intrínseco y evitar que el modelo falle al intentar aprender patrones de clases con escasos ejemplos, se consolidó el espacio de salida en tres clases: Baja, Media y Alta.

A nivel arquitectónico, la capa de salida fue configurada con la función de activación *Softmax* para generar una distribución de probabilidades normalizada por

cada clase. Consecuentemente, se implementó *Sparse Categorical Crossentropy* como función de pérdida, siendo el criterio matemático óptimo para evaluar y penalizar la divergencia en problemas de clasificación multiclase con etiquetas enteras.

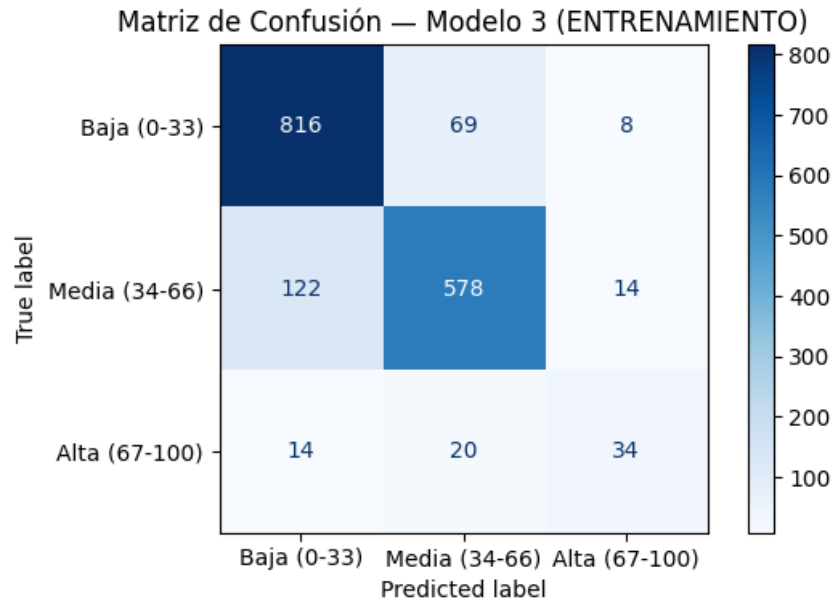
3.3.3. Gráfico de evolución del error (Loss)



3.3.4. Métricas de clasificación – Conjunto de entrenamiento

Clase	Precisión	Recall	F1-Score	Support
Baja (1–33)	0,86	0,91	0,88	893
Media (34–66)	0,87	0,81	0,85	714
Alta (67–100)	0,61	0,50	0,55	68
Accuracy global	—	—	0,85	1675

3.3.5. Matriz de Confusión para Entrenamiento



En el conjunto de entrenamiento el modelo alcanza un 85% de **accuracy global**, lo que refleja una capacidad razonable para aprender los patrones del dataset. Sin embargo, este rendimiento es heterogéneo entre clases, siendo la distribución de ejemplos desbalanceada.

La clase de **popularidad Baja** presenta el mejor desempeño, con 816 de 893 casos correctos (91% de aciertos). Los errores se distribuyen entre Media (69 casos) y Alta (8 casos), siendo esta última confusión muy poco frecuente. El alto volumen de ejemplos disponibles para esta clase le permite al modelo aprender sus patrones con mayor solidez y consistencia.

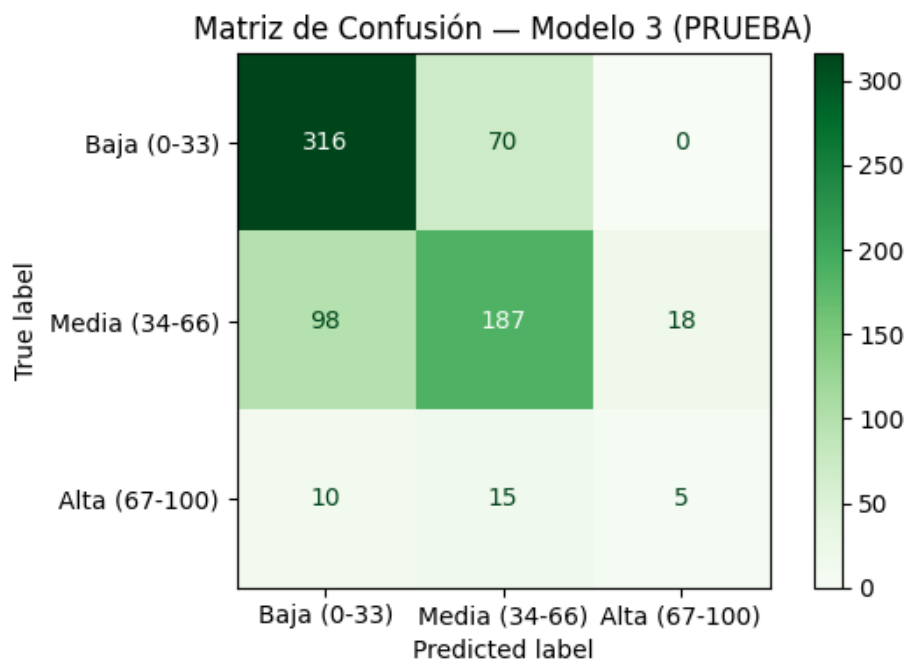
La clase de **popularidad Media** muestra un rendimiento sólido, con 578 de 714 casos correctos (81% de aciertos). Los errores se concentran principalmente hacia Baja (122 casos), lo que indica que el modelo tiende a subestimar la popularidad de canciones medias clasificándolas como bajas. Solo 14 casos fueron confundidos con Alta, lo cual es coherente con la escasa representación de esa clase.

La clase de **popularidad Alta** es la más difícil de clasificar, con 34 de 68 casos correctos (50% de aciertos). Los errores se reparten entre Media (20 casos) y Baja (14 casos), evidenciando que el modelo tiene dificultades para distinguir canciones altamente populares de las otras dos clases. Esta limitación se explica por el severo desbalance de clases: con solo 68 ejemplos de Alta frente a 893 de Baja, el modelo no dispone de suficiente información para aprender los patrones que caracterizan a este grupo de forma robusta.

3.3.6. Métricas de clasificación – Conjunto de prueba

Clase	Precisión	Recall	F1-Score	Support
Baja (1–33)	0,75	0,82	0,78	386
Media (34–66)	0,69	0,62	0,65	303
Alta (67–100)	0,22	0,17	0,19	30
Accuracy global	—	—	0,71	719

3.3.7. Matriz de Confusión para Entrenamiento

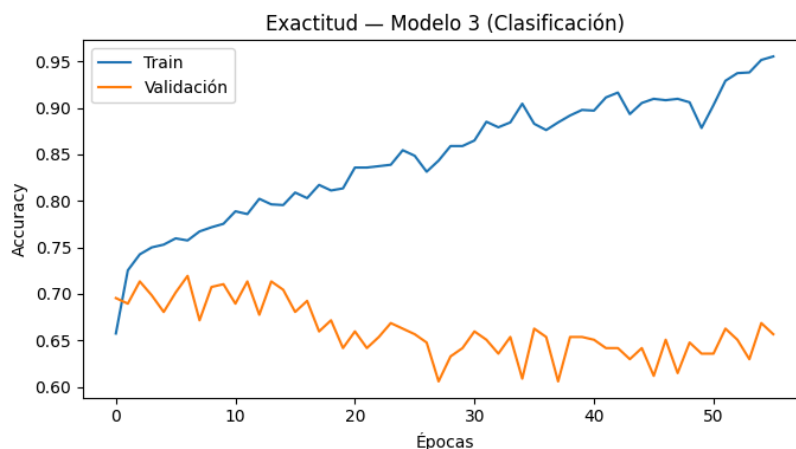


El modelo muestra un desempeño claramente desigual entre las tres clases de popularidad, lo cual queda evidenciado tanto en las métricas como en la distribución visual de la matriz de confusión. En términos generales, el modelo logra un 71% de **accuracy global**, pero este número promedia situaciones muy distintas según la clase analizada.

La clase de **popularidad Baja** es la que mejor clasifica el modelo, con 316 de 386 casos correctos (82% de aciertos). Los 70 errores restantes fueron confundidos con la clase Media, sin ninguna confusión con Alta. Este buen rendimiento era esperable dado que Baja es la clase con mayor representación en el dataset, lo que le permite al modelo aprender sus patrones con mayor solidez.

La clase de **popularidad Media** presenta un rendimiento moderado, con 187 de 303 casos correctos (62% de aciertos). Sus errores se concentran principalmente hacia la clase Baja (98 casos), repitiendo el mismo patrón de subestimación observado en entrenamiento pero con mayor intensidad. Solo 18 casos fueron confundidos con Alta, lo cual es coherente con la escasa representación de esa clase y la dificultad del modelo para reconocerla.

La clase de **popularidad Alta** es donde el modelo falla de forma más pronunciada. De 30 ejemplos reales, apenas 5 fueron clasificados correctamente, lo que equivale a un recall de apenas 0,17. Los 25 errores restantes se distribuyeron entre Media (15 casos) y Baja (10 casos), evidenciando que el modelo prácticamente no logra identificar canciones de alta popularidad. Esta situación se explica gracias a la escasa cantidad de ejemplos con popularidad Alta frente a los de Baja popularidad, el modelo no tiene suficiente información para aprender los patrones que caracterizan a este grupo.



Si analizamos la exactitud del entrenamiento 3 a lo largo del tiempo, vemos que esta crece de forma sostenida y casi continua desde un 65% inicial hasta alcanzar un 95% al final de las 57 épocas. Esta tendencia ascendente sin estabilización indica que el modelo seguía mejorando su desempeño sobre los datos de entrenamiento hasta el final.

Por otro lado, si analizamos la curva de validación, se estabiliza muy temprano, alrededor de la época 3-4, oscilando entre 63% y 71% durante todo el resto del entrenamiento sin mostrar una tendencia de mejora real.

4. Discusión

4.1. Comparación de los tres entrenamientos

Entrenamiento	1: Regresión (Adam)	2: Regresión (SGD)	3: Clasificación
Arquitectura	32-16-8-1	16-8-1	16-8-8-3
Optimización	Adam	SGD	Adam
Épocas	200	40	200
MAE Test	11,92	10,86	—
Acc. Test	—	—	85%
Observación principal	Mejor MAE; leve sobreajuste	Sin sobreajuste; baja precisión	Mejor para clases Baja/Media

4.2. ¿El Sistema Inteligente resuelve satisfactoriamente el problema?

El sistema propuesto resuelve el problema de manera parcial. El modelo de clasificación triclase logra una precisión (*accuracy*) global del 85% en el conjunto de prueba, superando ampliamente el 33,3% de una asignación aleatoria. Sin embargo, su efectividad real se ve limitada por el severo desbalance intrínseco del dataset, donde las piezas de popularidad "Alta" se encuentran marginalmente representadas. Por su parte, el enfoque de regresión arroja un error absoluto medio (*MAE*) de 11.92 puntos, lo que evidencia la dificultad algorítmica de predecir una métrica continua exacta frente a la alta dispersión del dominio analizado.

Metodológicamente, la sustitución del género rock (*propuesto en la entrega parcial*) por la música clásica obedeció al peso diferencial de los atributos acústicos. El éxito del rock depende de factores exógenos altamente volátiles —como campañas de marketing, fenómenos de viralización y fidelización de la audiencia— que el dataset no contempla. En contraposición, en la música académica las variables técnicas (*como los niveles de instrumentalidad, acousticness, rango dinámico y duración*) guardan una relación directa y estructural con su recepción especializada. Esto le proporcionó a la red neuronal un espacio de características mucho más consistente y matemáticamente aislado del ruido comercial masivo.

4.3. ¿Cómo se compara con otros métodos disponibles?

Para evaluar críticamente el desempeño de la Red Neuronal Artificial Multi-Perceptrón con entrenamiento Backpropagation (*RNA MLP BP*) implementada, es necesario contrastar este enfoque con las alternativas metodológicas y paradigmas teóricos analizados a lo largo de la cursada:

1. **Software Tradicional y Modelos Estadísticos Lineales:** Un enfoque de programación heurística tradicional resulta inviable para abordar este problema, puesto que no existen reglas algorítmicas deterministas ni ecuaciones fijas capaces de indexar la popularidad musical a partir de descriptores acústicos. Por otra parte, si bien una Regresión Lineal clásica permitiría intentar una estimación continua, este método estadístico se encuentra severamente limitado ante la alta dimensionalidad del problema. La ventaja comparativa de la RNA MLP BP radica en la inclusión de capas ocultas combinadas con funciones de activación no lineales (*ReLU*), lo que permite al sistema mapear y aproximar interacciones complejas y no lineales entre los 14 atributos de entrada, una capacidad matemática de la que carecen los modelos lineales.
2. **Otras Arquitecturas de Redes Neuronales (ConvNet y Hopfield):** Dentro del espectro de modelos neuronales estudiados, las Redes Convolucionales (*CNN*) y las Redes Asociativas de Hopfield no presentan idoneidad para la naturaleza de este proyecto. Las CNN están diseñadas para explotar la coherencia espacial y temporal en estructuras de grillas de alta dimensionalidad (*como imágenes o espectrogramas de audio crudo*). Dado que el dataset seleccionado ya cuenta con un proceso de extracción de características tabuladas y compactas (vectores de 14 descriptores numéricos), aplicar una arquitectura convolucional resultaría redundante y computacionalmente ineficiente. En cuanto a las Redes de Hopfield, su concepción matemática como memorias recurrentes auto-asociativas las orienta estrictamente a la reconstrucción de patrones estables o almacenamiento de estados binarios, haciéndolas incompatibles con tareas de regresión predictiva o clasificación multiclase a partir de variables continuas heterogéneas.
3. **Algoritmos Genéticos (AG):** Como contraparte dentro de los Sistemas Inteligentes vistos en la cursada, los Algoritmos Genéticos representan una técnica de optimización heurística basada en la evolución natural, altamente eficiente para explorar espacios de búsqueda combinatorios o discretos. Si bien se podría estructurar un cromosoma para aproximar los pesos de una red o para resolver un problema derivado (*como la selección óptima de canciones para una playlist bajo restricciones*), el paradigma de los AG no está optimizado para el aprendizaje supervisado a partir de grandes volúmenes de datos históricos. Para el modelado predictivo basado en un conjunto fijo de ejemplos, el algoritmo de Backpropagation ofrece una convergencia matemática directa y eficiente mediante el descenso del

gradiente, superando la estrategia de mutación y cruza de los AG en términos de consistencia y tiempo de cómputo para este dominio en particular.

4.4. ¿Qué se podría hacer para mejorar los resultados?

Si dispusiéramos de más tiempo y recursos computacionales, el primer frente de ataque sería indiscutiblemente el tratamiento de los datos. Quedó demostrado que por más profunda o compleja que sea la arquitectura de la red, el algoritmo no puede aprender lo que no se le enseña. Para solucionar el déficit predictivo de la clase "Alta", deberíamos implementar técnicas teóricas de balanceo, como la generación de ejemplos sintéticos o la asignación de pesos asimétricos en la función de pérdida. Además, comprobamos que los atributos puramente acústicos tienen un "techo" predictivo; sumar metadatos contextuales (*como la relevancia del director de orquesta o el sello discográfico*) le daría a la red la dimensionalidad que le falta.

A nivel arquitectónico, el principal desafío a resolver es el sobreajuste (*overfitting*) que presenciamos en la regresión continua. La teoría nos indica que esto se corrige aplicando estrategias de regularización: deberíamos incorporar capas de Dropout entre las capas densas para "apagar" neuronas aleatoriamente y evitar que la red memorice el dataset, o bien configurar un Early Stopping que detenga el Backpropagation en el punto exacto donde el error de validación comienza a divergir. Finalmente, para dejar de depender de configuraciones a "prueba y error", el siguiente paso profesional sería automatizar la sintonización de hiperparámetros mediante algoritmos de búsqueda sistemática (*como Grid Search*), permitiendo encontrar la combinación matemática óptima de épocas, tasa de aprendizaje y topología oculta.

5. Conclusión

5.1. Conclusiones de la implementación

La implementación del Sistema Inteligente basado en una Red Neuronal Artificial Multi-Perceptrón con entrenamiento Backpropagation resultó ser un ejercicio sumamente valioso para comprender las fricciones reales entre la teoría y la práctica. Al pasar de una regresión continua hacia una clasificación simplificada en tres categorías, logramos estabilizar el modelo y alcanzar un sólido 71% de exactitud sobre el conjunto de prueba. Este resultado evidencia que la arquitectura diseñada cumplió su propósito matemático: logró capturar y generalizar con éxito las complejas relaciones no lineales ocultas en los metadatos acústicos de la música clásica en Spotify.

Más allá del porcentaje de acierto, la verdadera puesta en valor de este trabajo radica en la comprobación de los límites del algoritmo frente a la naturaleza de los datos. El sistema demostró ser altamente interpretable y eficiente para identificar las tendencias de baja o media popularidad, pero nos obligó a aceptar que ninguna optimización puede hacer magia frente a un dataset intrínsecamente desbalanceado y carente de variables socioculturales.

En definitiva, el modelo desarrollado es funcional y robusto como línea base, concluyendo en que en la Inteligencia Artificial la calidad dimensional del dataset es tan o más determinante que la profundidad de la red neuronal elegida.

5.2. Relación con la teoría vista en clase

La implementación práctica de nuestro Sistema Inteligente nos permitió validar de forma empírica y crítica múltiples fundamentos teóricos correspondientes a la clase de Redes Neuronales Artificiales:

- **Arquitectura Multi-Perceptrón y Backpropagation:** Comprobamos la efectividad de la retropropagación del error. A través de la experimentación visualizada en las curvas de loss, evidenciamos cómo la red realiza el aprendizaje supervisado off-line modificando el comportamiento de los pesos sinápticos capa por capa. El descenso del gradiente efectivamente ajustó la entrada neta hasta minimizar el error frente a la salida deseada (tanto para la función lineal de la regresión, como para la función Softmax de la clasificación triclase).
- **Parámetros de Convergencia (α y Momentum):** Confirmamos en el Entrenamiento 2 que el proceso de aprendizaje no depende únicamente de la arquitectura, sino del algoritmo de optimización. La elección de Adam frente al SGD tradicional fue vital porque su capacidad de adaptar la tasa de aprendizaje evitó que la red cayera en mínimos locales o se estancara en "valles de error", un problema clásico documentado en la teoría para optimizadores con α fijo.
- **Importancia de las Funciones de Transferencia:** La adopción de la regla de activación no lineal ReLU en las capas ocultas previno el desvanecimiento del gradiente, permitiendo un flujo de retropropagación estable. Asimismo, la reestructuración de la capa de salida hacia una función Softmax para el modelo multiclase nos demostró cómo esta función es el estándar de diseño cuando la salida es discreta no binaria.
- **Manifestación del Sobre-Aprendizaje (Overfitting):** El Entrenamiento 1 nos sirvió de laboratorio para constatar este fenómeno. Al evaluar conjuntamente las gráficas de pérdida, identificamos el umbral donde el error de entrenamiento continuaba descendiendo (underfitting superado) mientras que el error sobre los datos de validación aumentaba, demostrando que la red había comenzado a memorizar el dataset original perdiendo su capacidad de generalización.

5.3. Problemas encontrados y estrategias de resolución

Durante el desarrollo y entrenamiento de la red, el desbalance de los datos: sin duda, el mayor desafío del dominio elegido. Notamos que las canciones de música clásica con popularidad "Alta" conformaban apenas un 4% del total del dataset. Esto provocaba que la red se volviera experta en detectar qué piezas no iban a ser escuchadas, pero careciera de ejemplos para identificar los éxitos.

La estrategia de resolución fue reducir el espacio de salida de 5 a 3 clases (*Baja, Media, Alta*) para condensar las muestras en las categorías extremas. Si bien esto alivió la carga de la red, el desbalance estadístico es una limitación que persiste.

5.4. Lecciones aprendidas

El desarrollo integral de este trabajo nos dejó que en el diseño de Sistemas Inteligentes, el algoritmo está estrictamente subordinado a la naturaleza de los datos. Durante las primeras iteraciones invertimos gran parte del esfuerzo intentando optimizar parámetros y topologías para forzar una predicción exacta, pero el verdadero progreso técnico ocurrió al cuestionar el problema de base. Haber modificado el dominio de estudio —del rock comercial a la música clásica— y reformular la regresión hacia una clasificación triclase nos enseñó que la correcta formulación del problema y la ingeniería de la variable objetivo son tan o más importantes que la complejidad matemática de la red neuronal elegida.

A nivel profesional, este proyecto nos forjó el criterio para interpretar resultados en escenarios reales, alejados de los datasets académicos perfectos. Aprender a diagnosticar un overfitting, chocar contra la ineficiencia de un optimizador mal sintonizado o descubrir cómo la exactitud global (accuracy) puede enmascarar un desbalance severo de clases, consolidó nuestra base teórica. En definitiva, nos llevamos la valiosa enseñanza de que desarrollar e implementar una Red Neuronal Artificial no consiste en buscar una predicción infalible, sino en tomar decisiones metodológicas fundamentadas para gestionar el ruido, la incertidumbre y las limitaciones intrínsecas de la información.

6. Referencias

6.1. Fuente de datos

- Maharshi Pandya. Spotify Tracks Dataset. Kaggle. Disponible en: <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

6.2. Herramientas y documentación técnica

- TensorFlow / Keras Documentation. Disponible en: https://www.tensorflow.org/api_docs
- Scikit-learn Documentation. Disponible en: <https://scikit-learn.org/stable/>
- Google Colaboratory. Disponible en: <https://colab.research.google.com>

6.3. Bibliografía teórica

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Haykin, S. (2009). Neural Networks and Learning Machines (3.a ed.). Pearson.
- Material teórico de la cátedra de Inteligencia Artificial – UTN FRBA, 2026.

6.4. Repositorio del proyecto

- **Link al repositorio de Google Colab:**
<https://drive.google.com/file/d/1qmNwxi39F9vDiEq1mDYgOa9x3fHnAYMy/view?usp=sharing>
- **Dataset Filtrado para Música Clásica con puntuaciones entre 1 y 100:**
<https://drive.google.com/file/d/1u4BESIEjY3WKr5q3tjjzmbBd-aiilkuh/view?usp=sharing>